SOFTWARE QUALITY ASSURANCE SPECIFICATION

LIGHTWEIGHT SOFTWARE TEST PLAN (STP)

Target Application: Automation Exercise (E-Commerce Web Application)
Strategy, ISTQB Test Techniques, Execution Gates, and Defect Governance

Document Identifier	STP-2026-AE01
Author & Role	Senior QA Engineer / QA Lead
Document Version / State	Version 1.0 (Final Approved Deliverable)
Target System Under Test (SUT)	https://automationexercise.com (Live Demo Web Store)
Creation & Baseline Date	July 3, 2026 (Updated October 2026)
Associated Workbooks	AutomationExercise_Test_Suite.xlsx (54 TCs / 16 Defects)

1. Executive Summary & Testing Objectives

This Lightweight Software Test Plan establishes the formal Quality Assurance strategy for the Automation Exercise web platform. As an end-to-end e-commerce store, the system encompasses user onboarding, product catalog discovery, cart state management, checkout billing calculations, and backend RESTful service transactions.

The primary objective of this testing cycle is to ensure business process integrity and safeguard customer experience across critical revenue funnels. Rather than bloated, outdated 50-page specifications, this lightweight document emphasizes high-impact testing, empirical ISTQB design techniques, strict quality gates, and transparent defect accountability.

> **📌 Senior QA Quality Philosophy**
> Quality is not tested in; it is designed in. Our testing strategy balances structured black-box specifications (Equivalence Partitioning, Boundary Value Analysis) with adversarial error guessing (SQL injection, URL tampering, negative cart balances) to catch catastrophic production bugs early.

2. Scope of Testing

A rigorous definition of In-Scope and Out-of-Scope boundaries ensures optimal resource allocation and prevents scope creep while maintaining laser focus on core user journeys.

Module / Testing Area	Scope	Senior Technical Rationale & Boundaries
User Registration & Profile	**IN-SCOPE**	Critical entry funnel: form validations, duplicate email checks, data persistence, and secure account deletion.
Authentication & Session	**IN-SCOPE**	Security baseline: login/logout flows, password masking, remember-me cookies, and unauthorized route guards.
Product Catalog & Search	**IN-SCOPE**	Discovery engine: exact/partial text search, URL encoding handling, brand facets, and mobile responsive menus.
Shopping Cart Engine	**IN-SCOPE**	State lifecycle: dynamic item addition, negative quantity rejection, zero-stock checks, and line-item recalculation.
Checkout & Financial Billing	**IN-SCOPE**	Revenue-critical: address mapping, promo discount computation, tax rate precision, and credit card format handling.
Contact Us & Newsletter	**IN-SCOPE**	Support & leads: file attachment extension filtering (.exe blocking) and newsletter AJAX subscription.
Backend REST APIs	**IN-SCOPE**	Data contract validation: 14 Postman requests verifying HTTP status codes (200/400/404) and JSON schemas.

2.1 Out-of-Scope Activities & Explicit Justification

Excluded Activity	Engineering & Business Rationale
Live Payment Gateway Transactions	Real credit card processing (Stripe/PayPal live APIs) is excluded; PCI-DSS tokenization compliance is out of scope for a sandbox testing store. Mock credentials used.
High-Concurrency Stress Testing (>10k VUs)	Target SUT is hosted on shared cloud infrastructure without dedicated autoscaling clusters. High-volume load testing would trigger infrastructure DoS blocks.
Deep Infrastructure Penetration Attacks	Hardware-level DDoS, memory buffer overflows, and network port scanning are excluded. Security testing is strictly limited to application-layer input validation (SQLi, XSS, MIME).

3. Test Strategy & Applied ISTQB Test Techniques

To showcase enterprise-grade QA competence, test cases are not written haphazardly. Each test suite systematically applies formal Black-Box and Experience-Based Test Design Techniques aligned with ISTQB Certified Tester Advanced Level (CTAL) guidelines:

Test Technique	Theoretical Foundation	Concrete Application in Project
Equivalence Partitioning (EP)	Divides input domain into valid and invalid partitions where all members are assumed to be processed identically.	• Email validation: Valid format ('user@domain.com') vs. Missing '@' ('userdomain.com') vs. Duplicate DB email. • File uploads: Valid doc types (.pdf, .jpg) vs. Executable binary partition (.exe) -> Uncovered BUG-016.
Boundary Value Analysis (BVA)	Tests boundaries of equivalence partitions (min, min-1, min+1, max, max-1, max+1) where software failure density is highest.	• Password minimum length: 5 chars (Reject), 6 chars (Accept). • Cart Quantity: 0 & -2 (Negative boundary -> Uncovered BUG-008). • Expiration year: 2-digit '28' vs. 4-digit '2028' (Uncovered BUG-013).

Decision Table Testing	Models complex business logic where outcomes depend on combinations of multiple conditional inputs.	• Checkout authorization & promo matrix: [User Logged In?] x [Cart Items > 0?] x [Valid Promo Code?] -> Validates discount math ($100 - 10% = $90). Uncovered BUG-011 (100% deduction).
State Transition Testing	Validates system behavior and state transitions based on valid and invalid sequence events.	• User Session Lifecycle: Anonymous Guest -> Authenticated -> Checkout In-Progress -> Order Placed -> Logged Out. • Back-button browser cache invalidation after logout.
Error Guessing & Exploratory	Leverages tester intuition, historical defect patterns, and adversarial thinking to uncover unanticipated flaws.	• SQL Injection vectors in name input (John' OR '1'='1 -> Uncovered BUG-002). • URL encoding special characters ('%' in search query -> Uncovered BUG-004). • Double-click delete race condition -> Uncovered BUG-009.
Cross-Browser Compatibility	Verifies functional parity and responsive UI stability across varying rendering engines (Blink, Gecko, WebKit).	• Chrome, Firefox, Edge, and iOS Safari tested. • Uncovered Firefox-specific disabled checkout button (BUG-012) and iOS Safari accordion touch failure (BUG-005).
Automated API Verification	Validates backend microservices without UI overhead, asserting response status codes, payload structures, and response time.	• 14 Postman API requests with 43 JavaScript test assertions covering /api/productsList, /api/verifyLogin, and /api/getUserDetailByEmail.

4. Test Environment & Tooling Ecosystem

A multi-tiered testing environment was configured to ensure high fidelity with real-world customer hardware and browser configurations.

Tool / Environment Component	Technical Specification	Role in Testing Cycle
Production SUT Web URL	https://automationexercise.com	Target live application under test for end-to-end user journey validation.
Backend API Base URL	https://automationexercise.com/api/	Target endpoints for headless Postman API test assertions.
Desktop Browsers	Google Chrome v120+, Mozilla Firefox v120+, Microsoft Edge v120+	Cross-browser matrix covering Chromium (Blink) and Gecko engines.
Mobile Viewports	iOS Safari (iOS 17.2) / Chrome DevTools Mobile Emulation (390x844)	Touch interaction, responsive accordion navigation, and small-screen layout checks.
Postman API Client	Postman v11.x with JavaScript Assertion Runner	Automated contract testing, schema verification, and HTTP status assertions.
Chrome DevTools	Console, Network tab (XHR/Fetch), DOM Inspector, Storage tab	Root Cause Analysis: inspecting session cookies, console error tracebacks, and API payloads.

5. Quality Gates & Decision Criteria

To establish objective governance, the testing lifecycle enforces strict quantitative Quality Gates:

Quality Gate Stage	Measurable Criteria & Enforcement Policy
Entry Criteria (Start Testing)	1. SUT application URL is reachable with HTTP 200 response. 2. Smoke test suite (Login, Product View, Add to Cart) passes 100%. 3. Test Plan approved and Master Test Cases Matrix seeded with test data accounts.
Suspension & Resumption Criteria	1. Suspended if core Signup or Payment gateway returns unhandled 500 error on >30% requests. 2. Suspended if testing database is wiped or corrupted. 3. Resumed only after hotfix deployment and QA smoke verification.
Exit Criteria (Release Sign-off)	1. 100% of designed test cases executed (54/54). 2. Zero (0) Critical or High (P1) defects remain unaddressed without executive waiver. 3. 100% of failing test cases mapped to documented defects with Root Cause Analysis. 4. All 14 Postman API collection requests pass with 100% assertion success.

6. Defect Management Lifecycle & SLA Governance

Defects are tracked throughout their lifecycle using a standardized Kanban workflow modeled after enterprise Jira boards: New -> Open -> In Progress -> Resolved -> QA Retest -> Closed (or Reopened).

Severity Level	Priority	Impact Definition & System Criteria	Target Resolution SLA
Critical (S1)	High (P1)	System crash, database SQL error exposure, financial loss (free order exploit BUG-011, negative cart balance BUG-008).	< 4 Hours (Immediate Hotfix)
High (S2)	High (P1 / P2)	Major business function failure without workaround (Firefox checkout disabled BUG-012, zero-stock purchase BUG-007, executable file upload BUG-016).	< 24 Hours
Medium (S3)	Medium (P2)	Non-critical feature flaw with available workaround (session cookie expiration BUG-003, cart badge delayed update BUG-010, tax calculation disparity BUG-014).	Current Sprint
Low (S4)	Low (P3)	Cosmetic flaw, double-click button delay (BUG-009), raw HTML email tags (BUG-015), or minor navigation redirect inconsistency (BUG-006).	Future Release Backlog

7. Test Deliverables & Traceability Artifacts

All testing work products are version-controlled and cross-linked to form a complete bidirectional Requirement Traceability Matrix (RTM):

Deliverable Item	Format & Repository Path	Description & Stakeholder Audience
Software Test Plan (STP)	Word (.docx) & PDF (docs/ & test-artifacts/)	Lightweight strategic guide detailing scope, techniques, and governance for tech leads.
Master Test Cases Matrix	Excel (.xlsx) & CSV (testcases/ & test-artifacts/)	54 fully structured test cases with priority, test data, steps, actual results, and bug IDs.
Defect Management Log	Excel Sheet & Markdown (testcases/ & test-artifacts/)	16 logged bugs with Severity, Priority, Steps to Reproduce, and Root Cause Analysis.
Postman API Test Suite	JSON Collection & Environment (api-testing/)	14 automated API requests asserting backend status codes, headers, and payload schemas.
Defect Evidence Repository	PNG Screenshot Evidence (evident/bugs/)	High-resolution proof captures indexed by Bug ID validating actual defect behavior.

8. Risk Assessment & Mitigation Strategies

Proactive identification of testing risks ensures business continuity and prevents cycle blockages:

Identified Risk	Likelihood	Impact	Mitigation Action & Senior QA Protocol
Third-Party Google Ads UI Overlays	High	Medium	External ad banners frequently obscure modal dialogs and Add to Cart buttons. Mitigation: Configured Chrome DevTools uBlock Origin testing profile to isolate application code from ad network flakiness.
Public Database Test Data Overwrite	Medium	High	Shared public testing environment permits external users to register existing emails or alter catalog states. Mitigation: Implemented dynamic timestamped email addressing (e.g., test.qa+<timestamp>@test.com) for idempotent test runs.
Transient Network Latency & API Spikes	Low	Medium	Slow network responses causing false-positive timeouts. Mitigation: Set Postman request timeouts to 8000ms with retry policies and verified backend health via DevTools Network payloads.

9. Document Approvals & Governance Sign-off

By signing below, stakeholders acknowledge and approve the test strategy, scope, entry/exit criteria, and governance procedures defined herein.

Role / Title	Stakeholder Name	Approval Status	Sign-off Date
Lead QA Engineer	Senior QA Specialist	APPROVED	July 3, 2026
Lead Software Architect	Principal Engineer	APPROVED	July 4, 2026
Frontend Team Lead	Senior UI Developer	APPROVED	July 4, 2026
Product Delivery Manager	VP of Engineering	APPROVED	July 5, 2026